HOUSE PRICE PREDICTION

A Beginner-Friendly Machine Learning Project

Project type	Software-only / Machine Learning
Main language	Python
Suggested interface	Streamlit web app
Level	Beginner to intermediate

Prepared as a project guide with problem statement, objectives, methodology, implementation, sample code, testing, and future scope.

2. Abstract

House Price Prediction is a machine learning project that estimates the likely price of a house using details such as area, number of bedrooms, number of bathrooms, location-related features, and house age. Instead of guessing manually, the system learns patterns from previously recorded house data and uses those patterns to estimate a price for a new house.

This project is intended for learning and demonstration. The estimate is not a certified property valuation and may not reflect current market prices unless the dataset is recent, local, and representative.

3. Introduction

House prices can change based on many factors. Two houses with similar sizes may have different prices because of their location, condition, nearby facilities, or market demand. Machine learning can study historical examples and learn how these factors relate to price.

In this project, a user enters house details in a simple web page. The trained model processes the details and displays an estimated price. The project covers data preparation, model training, model evaluation, and a basic user interface.

4. Problem Statement

People may find it difficult to estimate a reasonable house price because prices depend on several factors and information may be scattered. The aim is to build a software system that uses historical housing data to predict an approximate price from selected house features.

5. Objectives

- Understand how a machine learning regression problem works.
- Prepare a housing dataset for model training.

- Train and compare regression models.
- Evaluate prediction quality using suitable metrics.
- Build a simple interface for entering house details and viewing an estimate.
- Document limitations and possible future improvements.

1. Project Scope

- Software-only project; no special hardware is required.
- A public CSV dataset is used for training and testing.
- The first version predicts price from numeric and/or encoded categorical features available in the dataset.
- The demo can be run locally on a laptop.
- The project does not replace a professional property valuation.

1. Existing System and Proposed System

Existing approach

A person may compare listings, ask local agents, or estimate from personal experience. These approaches can be useful, but they may be inconsistent and may not consider many factors together.

Proposed approach

The proposed system uses a trained regression model. It accepts house features, applies the same preprocessing used during training, and returns an estimated price. Its quality depends on the dataset and how closely that dataset matches the target area and time period.

2. Requirements

Hardware

- Laptop or desktop computer.
- At least 4 GB RAM recommended.
- Internet connection for downloading the dataset and installing libraries.

Software

- Python 3.10 or newer.
- VS Code, PyCharm, or Jupyter Notebook.
- Libraries: pandas, numpy, scikit-learn, joblib, streamlit, matplotlib (optional).
- A housing dataset in CSV format.

1. Dataset and Features

For a beginner project, use a public housing dataset such as the California Housing dataset available through scikit-learn, or a house-sales CSV dataset from a trusted public repository. Check the dataset licence, feature definitions, units, and date before use. The California Housing dataset is based on California districts, so it should not be presented as a direct estimate for Indian cities.

Example input features

Feature	Meaning
Area / living area	Approximate size of the house, using the dataset's stated unit
Bedrooms	Number of bedrooms, if available
Bathrooms	Number of bathrooms, if available
House age	Age of the property, if available
Location	Area, postcode, or geographic feature, if available
Target: price	Recorded sale/listing price; the model learns to predict this

Important: only use features that actually exist in the chosen dataset. Do not invent missing values or assume all datasets have the same columns.

2. Methodology

- Collect: download a suitable housing dataset.
- Inspect: check columns, data types, missing values, duplicates, and unusual values.
- Prepare: separate input features (X) from target price (y); encode categories if needed.
- Split: divide data into training and test sets, for example 80% and 20%.
- Train: fit a baseline model such as Linear Regression, then compare with a tree-based model such as RandomForest Regressor.
- Evaluate: use MAE, RMSE, and R^2 on the held-out test set.
- Save: store the selected trained model and preprocessing pipeline.
- Deploy: load the saved pipeline in a smallStreamlit app and display the estimate.

1. System Architecture

User enters house details
| v Input validation
| v Preprocessing pipeline
(missing values / encoding / scaling ifneeded) | v
Trained regression model | v Predicted house
price | v Display estimate and limitations

The preprocessing steps used for new inputs must match the steps used while training. A scikit-learn Pipeline or ColumnTransformercan help keep these steps consistent.

2. Step-by-Step Implementation

Step 1: Create a project folder

```
house_price_prediction / ■■■ data/
■   ■■■ housing.csv
■■■ models/
■   ■■■ house_price_ model.joblib
■■■ train_model.py
■■■ app.py
■■■ requirements.txt
■■■ README.md
```

Step 2: Install the libraries

```
python -m  venv  .venv  # Activate the
environment, then run:
pip install pandas numpy scikit-learn joblib streamlit
```

Step 3: Choose the dataset

Download a dataset and place it at data/housing.csv. Open it and identify the target price column and input columns. The sample code below assumes the target column is named 'price'; change that name and the feature list to match your CSV.

Step 4: Train and evaluate a model

```
# train_model.py from pathlib import Path import joblib import pandas as pd from
sklearn.model_selection import train_test_split from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline from sklearn.impute import SimpleImputer from
sklearn.preprocessing import OneHotEncoder , StandardScaler from sklearn.ensemble import
RandomForestRegressor from sklearn.metrics import mean_absolute_error , mean_squared_error ,
r2_score import numpy as np
DATA_PATH = Path("data/housing.csv") TARGET = "price "  # Change to the exact
target column in your CSV

df = pd.read_ csv(DATA_PATH) df =
df.dropna(subset=[TARGET]) X =
df.drop(columns=[TARGET]) y =
df[ TARGET]

numeric_cols = X.select_dtypes (include=["number"] ).columns.tolist() categorical_cols =
X.select_dtypes (exclude=["number"]  ).columns.tolist()

numeric_pipe = Pipeline([     ("imputer",
SimpleImputer(strategy="median")),
   ("scaler", StandardScaler()) ] ) categorical_pipe = Pipeline([     ("imputer",
SimpleImputer(strategy=" most_frequent")),
   ("onehot ", OneHotEncoder (handle_unknown ="ignore")) ] )

preprocess = ColumnTransformer([     ("num",
numeric_pipe, numeric_cols),
   ("cat", categorical_pipe, categorical_cols) ] )

model = RandomForestRegressor(     n_estimators=200,
random_state =42,  n_jobs =- 1 ) pipeline =
Pipeline([     ("preprocess", preprocess),
   ("model", model)
])

X_train , X_test , y_train , y_test  = train_test_split(
   X, y,  test_size =0.2,  random_state = 42 )
pipeline.fit(X_train , y_train) pred =
pipeline.predict(X_test )

rmse = np.sqrt(mean_squared_error (y_test , pred))  print( "MAE:",
round(mean_absolute_error (y_test , pred), 2))  print( "RMSE:", round(rmse, 2))
print( "R2:", round(r2_ score(y_test , pred), 3))

Path("models"). mkdir(exist_ok=True)  joblib.dump({"pipeline": pipeline, "columns":
X.columns.tolist()},            "models/house_price_model.joblib") print( "Saved model to
models/house_price_model.joblib")
```

The code is a template, not a one-click solution for every dataset. Confirm that the CSV has a numeric target price column, remove identifiers that would leak the answer, and adjust columns or data cleaning as required.

Step 5: Create a basic web application

```
# app.py import joblib
import pandas as pd import
streamlit as st

saved = joblib.load("models/house_price_model.joblib") pipeline =
saved["pipeline"] columns = saved["columns"]

st.title("House Price Prediction") st.write("Enter values using the same feature names and units as the training
dataset.")

# Generic text inputs work for a starter demo. # For a final project, replace these with appropriate
number/select widgets. values = {} for col in columns:     values[col] = st.text_input(f"{col}", value="")

if st.button("Predict price"):     if any(v.strip() == "" for v in
values.values()):
      st.error("Please fill in every field.")     else:
      row = pd.DataFrame([values], columns= columns)        # Convert columns
that were numeric during training.
      # A production version should save the training schema and        # validate/convert
each field according to that schema.        try:            for col in row.columns:
            row[col] = pd.to_numeric(row[col], errors="ignore ")          prediction = pipeline.predict(row)[0]
st.success(f"Estimated house price: {prediction:,.2f}")          st.caption("This is a model estimate, not an official valuation.")
except Exception as e:          st.error(f"Could not make a prediction. Check input values and data types. Details: {e}")
```

For a stronger implementation, save the numeric and categorical column lists from training and build matching number inputs and dropdowns in Streamlit. That avoids type mismatches and makes the interface easier to use.

Step 6: Run the application

```
python train_model.py  streamlit run
app.py
```

Streamlit prints a local URL, commonly http://localhost:8501. Open it in your browser while the app is running.

3. Evaluation Metrics

- MAE (Mean Absolute Error): the average absolute difference between actual and predicted prices. Lower is better.
- RMSE (Root Mean Squared Error): gives more weight to larger errors. Lower is better.
- R^2 (R-squared): compares the model with a simple average-based baseline; closer to 1 is generally better, but it can be negative.

Do not write made-up accuracy percentages. Train and test the model on your chosen dataset, then report the actual measured results, dataset split, and units.

1. Testing Plan

Test	Input / action	Expected result
Valid input	Enter valid feature values	A numeric estimate is shown
Missing input	Leave a required field blank	Helpful validation message appears

Unexpected category	Enter a category not seen during training	
Invalid number	Enter letters into a numeric field	App asks for a valid value or shows a clear error
Model file missing	Run app before training	Clear setup error; train model first

Pipeline handles it ifOneHotEncoder is configured as shown

2. Expected Outcome

The completed project should read housing data, train a regression model, measure its performance, save the model, and provide a simple interface that returns an estimated price for a new example. Actual performance depends on data quality, selected features, and how representative the dataset is.

3. Advantages

- Provides a consistent, data-based estimate.
- Demonstrates a complete machine learning workflow.
- Can be extended with location, property condition, and nearby facilities when reliable data is available.
- Can be built using open-source Python tools.

1. Limitations

- Predictions can be poor when the training data is old, small, biased, or from another region.
- Important factors such as property condition and local demand may be missing.
- A model may perform well on test data but poorly on real-world properties from a different market.
- The output is an estimate and must not be treated as a guaranteed selling price.

1. Future Enhancements

- Use a recent dataset from the target city or region.
- Add a map-based location feature if suitable data and privacy safeguards are available.
- Compare Linear Regression, Random Forest, and Gradient Boosting models.
- Add charts showing feature relationships and model errors.
- Improve input validation, user interface, and error messages.
- Deploy the application online with security, monitoring, and clear limitations.

1. Conclusion

House Price Prediction is a practical beginner machine learning project. It teaches data preparation, regression, model evaluation, and simple application development. The most important step is choosing a suitable dataset and honestly evaluating the model. With better local data and careful testing, the system can become a more useful decision-support tool.

2. References and Learning Resources

- Scikit-learn documentation: https://scikit-learn.org/
- Pandas documentation: https://pandas.pydata.org/docs/
- Streamlit documentation: https://docs.streamlit.io/
- California Housing dataset documentation:https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html